



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 9

Dr. Asad Mahmood

Feb. 04, 2026

Lecture Outline

- Announcements (Assignment and Quiz 1 Day/Time/Venue)
- Outline of Previous Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.5 Representing Instructions in the Computer
- Outline to Today's Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions

Chapter 2

Instructions: Language of the Computer

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - **2.5 Representing Instructions in the Computer**
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Representing Instructions

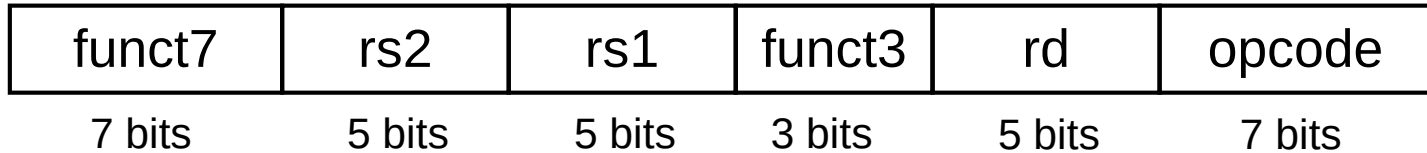
- There is a difference between the way humans instruct computers and the way computers see instructions.

add x9, x20, x21

0	21	20	0	9	51
0000000	10101	10100	000	01001	0110011
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Instructions are encoded in binary
 - Called machine code
- RISC-V instructions
 - Encoded as 32-bit instruction words
 - Field* and *instruction format*
 - Small number of formats
 - Simplicity favors Regularity!**

RISC-V R-format Instructions



■ Instruction fields

- opcode: Main operation (+format) of the instruction
- rd: destination register number
- funct3: 3-bit function code (additional opcode)
- rs1: the first source register number
- rs2: the second source register number
- funct7: 7-bit function code (additional opcode)

R-format Example

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

add x9, x20, x21

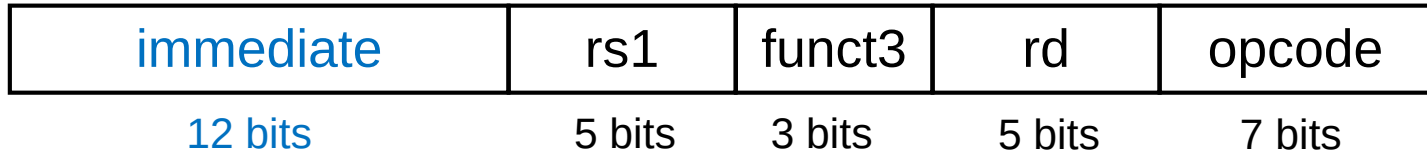
0	21	20	0	9	51
---	----	----	---	---	----

0000000	10101	10100	000	01001	0110011
---------	-------	-------	-----	-------	---------

0000 0001 0101 1010 0000 0100 1011 0011_{two} =

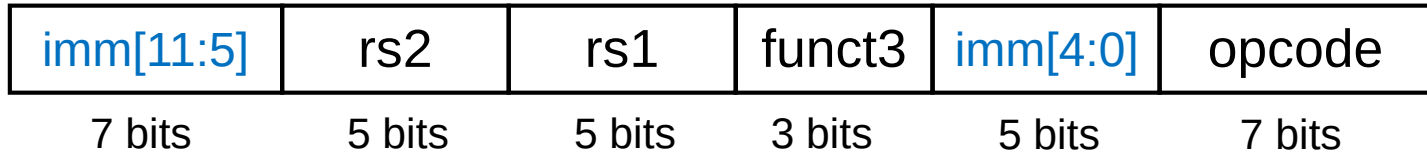
015A04B3₁₆

RISC-V I-format Instructions



- Immediate arithmetic and load instructions
 - rs1: source or base address register number
 - immediate: constant operand, or offset added to base address
 - 2s-complement, sign extended
- *Design Principle 3: Good design demands good compromises*
 - Different formats complicate decoding, but allow 32-bit instructions uniformly
 - Keep formats as similar as possible

RISC-V S-format Instructions



- Different immediate format for store instructions
 - rs1: base address register number
 - rs2: source operand register number
 - immediate: offset added to base address
 - Why Split?

RISC-V Opcodes for different Instruction Formats

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011
Instruction	Format	immediate		rs1	funct3	rd	opcode
addi (add immediate)	I	constant		reg	000	reg	0010011
lw (load word)	I	address		reg	010	reg	0000011
Instruction	Format	immed-iate	rs2	rs1	funct3	immed-iate	opcode
sw (store word)	S	address	reg	reg	010	address	0100011

RISC-V Instruction Formats

■ Example

immediate	rs1	funct3	rd	opcode
120	10	2	9	3
000011110000	01010	010	01001	0000011

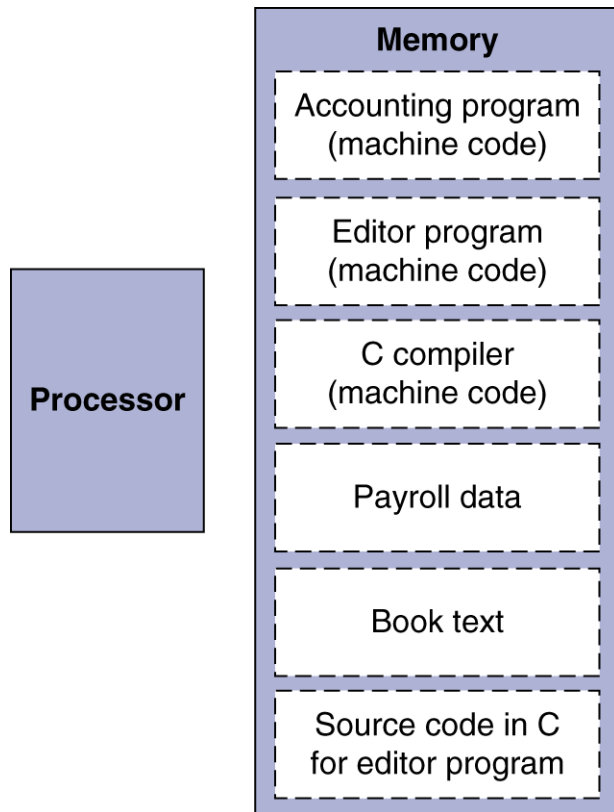
funct7	rs2	rs1	funct3	rd	opcode
0	9	21	0	9	51
0000000	01001	10101	000	01001	0110011

immediate	rs1	funct3	rd	opcode
1	9	0	9	19
000000000001	01001	000	01001	0010011

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
3	9	10	2	24	35
0000011	01001	01010	010	11000	0100011

Stored Program Computers

The BIG Picture



- Instructions represented in binary, just like data
- Instructions and data stored in memory
 - Harvard vs Von-Neumann Architecture
- Binary compatibility allows compiled programs to work on different computers
 - Standardized ISAs

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - **2.6 Logical Operations**
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

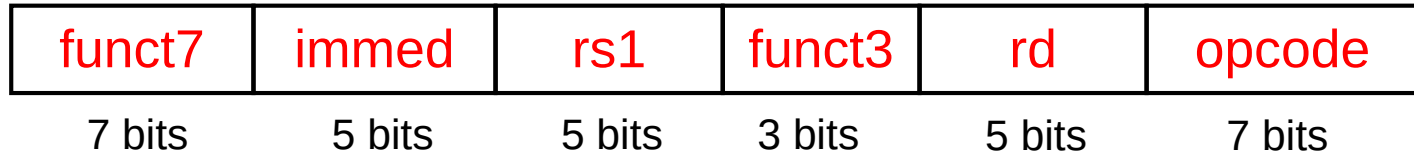
Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	RISC-V
Shift left	<<	<<	slli
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit XOR	^	^	xor, xori
Bit-by-bit NOT	~	~	

- Useful for extracting and inserting groups of bits in a word

Shift Operations



- **immed**: how many positions to shift
- **Shift left logical**
 - Shift left and **fill with 0 bits**
 - **$sll\ i$ by i bits multiplies by 2^i**
- **Shift right logical**
 - Shift right and fill with 0 bits
 - **$srl\ i$ by i bits divides by 2^i (unsigned only)**

AND Operations

- Useful to **mask bits in a word**
- and x9, x10, x11

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000
x9	00000000 00000000 00000000 00000000 00000000 00000000 00001100 00000000

OR Operations

- Useful to include bits in a word
- `or x9, x10, x11`

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000
x9	00000000 00000000 00000000 00000000 00000000 00000000 00111101 11000000

XOR Operations

- Differencing operation
- `xor x9, x10, x12`

x10	00000000	00000000	00000000	00000000	00000000	00000000	00001101	11000000
x12	11111111	11111111	11111111	11111111	11111111	11111111	11111111	11111111
x9	11111111	11111111	11111111	11111111	11111111	11111111	11110010	00111111

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - **2.7 Instructions for Making Decisions**
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Instructions for making decisions

- What distinguishes a computer from a simple calculator is its **ability to make decisions**.
- Otherwise, continue sequentially
- **beq rs1, rs2, L1**
 - if (rs1 == rs2) branch to instruction labeled L1
- **bne rs1, rs2, L1**
 - if (rs1 != rs2) branch to instruction labeled L1

Conditional Operations

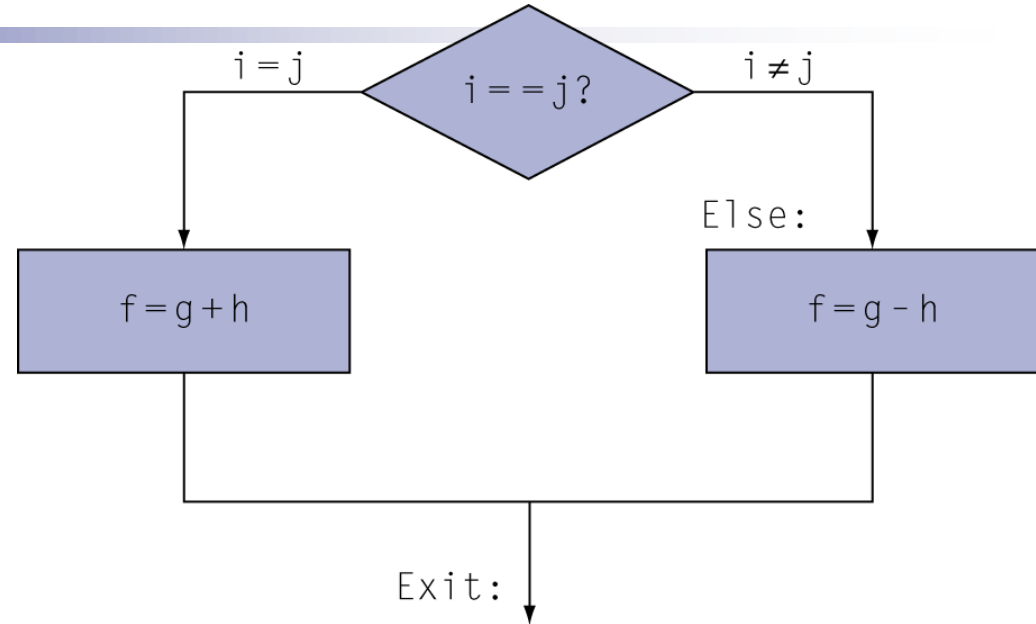
- What distinguishes a computer from a simple calculator is its ability to make decisions.
- Branch to a labeled instruction if a condition is true
 - Otherwise, continue sequentially
- `beq rs1, rs2, L1`
- `bne rs1, rs2, L1`

Compiling If Statements

- Example

C code:

```
if (i==j)
    f = g+h;
else
    f = g-h;
```



- f, g, h, i, j in $x19, x20, x21, x22, x23$
- Compiled RISC-V code?
- HLL better than Assembly as no labels/branches used
- Assembler relieves the compiler/assembly programmer from the tedious task of calculating branch addresses

Compiling Loop Statements

- C code:

```
while (save[i] == k)  
    i += 1;
```
- i in x22, k in x24, address of save in x25
- Compiled RISC-V code?